

Contents

Pendientes — Sistema de Gestión Logística	1
0. Pregunta resuelta (contexto)	1
1. Datos de ejemplo — simular un sistema con historia	1
2. Modo oscuro / modo claro	1
3. Cambio de paleta — verde oscuro	2
4. Sección de Auditoría — terminar de pulir	2
5. Selector de fecha (calendario) — mejorar la experiencia	2
6. Alertas en tiempo real (opcional — definir)	2
7. Testing — según la consigna oficial del TP	3
8. Otros requisitos de la consigna aún no cubiertos	3
9. Pendientes técnicos previos (ya identificados)	3

Pendientes — Sistema de Gestión Logística

Lista de trabajo pendiente. No incluye implementación, solo el detalle de qué hay que hacer.

0. Pregunta resuelta (contexto)

¿Las alertas pueden correr en tiempo real en localhost? Sí. `localhost` no es la limitación. Hoy la evaluación es **a demanda** (botón “Evaluar alertas”) por decisión de diseño, no por restricción técnica. La única condición para automatizarlas es que el backend esté corriendo. Ver tarea 6.

1. Datos de ejemplo — simular un sistema con historia

Ampliar el seed para que el sistema parezca llevar tiempo en uso, en vez de arrancar casi vacío.

- Muchos más viajes finalizados, distribuidos a lo largo de varios meses (para que el gráfico de “viajes por mes” del dashboard muestre una curva real y no un solo mes).
 - Más vehículos, con kilometrajes acumulados variados y coherentes con su antigüedad.
 - Historial de mantenimientos ya completados por vehículo (no solo pendientes).
 - Más choferes, con estadísticas acumuladas (viajes realizados, promedio de km) coherentes con su historial.
 - Documentación cargada para todos los choferes, con vencimientos variados (vigentes, por vencer, vencidos).
 - Registros de auditoría históricos, para que la pantalla de Auditoría no se vea vacía al inicio.
 - Alertas ya resueltas además de las pendientes, para que la pestaña “Resueltas” tenga contenido.
-

2. Modo oscuro / modo claro

- Implementar alternancia entre tema claro y oscuro en toda la aplicación.
 - Control visible para cambiar de modo (ej. en la barra superior).
 - Recordar la preferencia del usuario entre sesiones.
 - Verificar contraste y legibilidad en ambos modos, en todas las pantallas (tablas, diálogos, gráficos, chips de estado).
-

3. Cambio de paleta — verde oscuro

- Reemplazar el azul marino actual por una paleta de **verde oscuro** como color principal de la UI.
 - Aplicar en: sidebar, barra superior, botones primarios, chips/estados, gráficos y acentos.
 - Mantener coherencia con los colores semánticos (éxito, advertencia, error) para que sigan siendo distinguibles.
 - Debe funcionar tanto en modo claro como en modo oscuro (tarea 2).
-

4. Sección de Auditoría — terminar de pulir

- **Traducir al castellano todo lo que ve el usuario:** hoy se muestran los códigos internos tal cual (CREATE, UPDATE, VIEW_CREDENTIALS, USER, VEHICLE, TRIP, etc.). Deben mostrarse como texto legible (“Creación”, “Modificación”, “Consulta de credenciales”, “Usuario”, “Vehículo”, “Viaje”...). Aplica a la columna de acción, la de entidad y los filtros.
 - **Rediseñar el detalle de antes/después** con un enfoque más cercano al usuario:
 - Mostrar nombres de campo legibles en castellano en vez de las claves técnicas (licensePlate → “Patente”, isActive → “Activo”).
 - Formatear los valores (fechas en formato local, booleanos como “Sí/No”, estados traducidos).
 - Resaltar visualmente **qué campos cambiaron** en vez de mostrar dos bloques completos que el usuario tiene que comparar a ojo.
 - Ocultar o agrupar los campos que no cambiaron.
-

5. Selector de fecha (calendario) — mejorar la experiencia

Aplica a los filtros de **Reportes** y de **búsqueda de Viajes** (y cualquier otro filtro por fecha).

- El calendario actual (input nativo del navegador) resulta incómodo, especialmente al elegir mes y día.
 - Reemplazarlo por un selector de fecha propio con mejor navegación: cambio de mes/año ágil, selección clara del día.
 - Considerar un selector de **rango** (desde/hasta en un solo control) para los filtros que usan dos fechas.
 - Agregar atajos útiles si aplica (ej. “Hoy”, “Últimos 7 días”, “Este mes”).
 - Mantener el manejo correcto de zonas horarias ya resuelto (no debe reintroducir desfasajes).
-

6. Alertas en tiempo real (opcional — definir)

Hoy las alertas se evalúan solo al presionar el botón. Si se quiere que aparezcan solas, elegir uno de estos caminos:

- **Tarea programada en el backend:** un job que ejecute la evaluación cada X minutos mientras el servidor esté corriendo. Es lo más simple y suficiente para la demo.
- **Consulta periódica desde el frontend:** que la app recargue las alertas cada cierto tiempo. Simple, pero no genera alertas nuevas, solo refresca.
- **Notificaciones push (WebSockets / SSE):** verdadero tiempo real, con aviso inmediato al usuario. Es el más complejo.

Decisión pendiente: si se implementa y con cuál de las tres opciones.

7. Testing — según la consigna oficial del TP

Verificado en la consigna (<https://github.com/utnfrrods/tp>). Los requisitos exactos para **Aprobación Directa o en Examen** son:

Backend: - “Implementar **1 test automatizado por integrante**.” → con 3 integrantes, 3 tests automatizados. **Ya cumplido** (hay 23 tests unitarios). - “Implementar **1 test de integración**.” → **FALTA**. Los tests actuales son unitarios y corren sin base de datos. Hace falta al menos un test de integración que ejercite el flujo real contra la base (ej. crear un viaje, asignarlo y finalizarlo verificando los efectos en vehículo y chofer).

Frontend: - “Realizar al menos **1 test unitario de un componente**.” → **FALTA**. Los 5 tests actuales son de funciones utilitarias (fechas, rutas por rol), no de un componente renderizado. Hace falta testear un componente real (ej. un diálogo de formulario o la tabla), típicamente con Testing Library. - “Realizar al menos **1 test de end-to-end**.” → **FALTA**. No hay ningún test E2E automatizado. La guía E2E actual es manual. Hace falta uno automatizado (ej. con Playwright o Cypress) que recorra un flujo completo en el navegador.

Resumen de lo que falta en testing: 1 test de integración (backend), 1 test unitario de componente (frontend), 1 test E2E automatizado (frontend).

8. Otros requisitos de la consigna aún no cubiertos

Detectados al revisar la consigna oficial. No estaban en la lista original pero son necesarios para la entrega de Aprobación:

- **Documentación de la API de backend** — la consigna pide “Documentación de la API de backend (según la tecnología y standard utilizados)”. Hoy no existe. Habitualmente se resuelve con OpenAPI/Swagger.
 - **Deploy** — la consigna pide “Links de Deploy” y “Credenciales para utilizar la aplicación desplegada”. El sistema hoy corre solo en local.
 - **Video explicando el funcionamiento del sistema.**
 - **Evidencia del resultado de la ejecución de los tests automáticos** (capturas o salida de la corrida).
 - **Verificar los 3 breakpoints (SM, MD, LG)** y la estrategia mobile-first que exige la consigna de frontend.
 - **Documentación de gestión del proyecto** — metodología usada, minutas de reuniones, trackeo de tareas (la consigna lo pide explícitamente).
-

9. Pendientes técnicos previos (ya identificados)

- **Autocompletado de direcciones en modo estricto:** hoy sugiere direcciones reales pero permite texto libre; falta decidir si se bloquea el guardado sin selección. Requiere además habilitar la Places API en la key de Google.
- **Vulnerabilidades de dependencias:** dos reportadas por `npm audit` (esbuild/vite y react-router). Ambas requieren actualizaciones mayores; se dejaron para una pasada de endurecimiento previa al deploy. **No correr** `npm audit fix --force` sin planificarlo.